计算机学院实验报告

实验题目： 三种平面三角形点定位算法		学号：202200130077
日期：2024. 4. 09	班级：智能	姓名：薛润泽
Email：		

实验目的：

实现二维的 Catmull-Clark 细分方法

1，功能模块：数据录入，数据结构、细分算法、可视呈现；

2，数据录入：obj 格式，实现基本的文件读写功能；

3，数据结构：就是一个图结构，点和边分别存储，需要用到查询一邻域的方法；

4，细分算法：Catmull 细分方法；

5，可视呈现：基于 opencv 绘制线段，在二维屏幕上呈现细分的迭代过程，或其他任何可视 化手段（libigl, matlab, 等等都可以）

实验环境介绍：

Ubuntu

解决问题的主要思路：

Catmull-Clark 细分，主要思路是生成 face（面）点和 edge（边）点，生成完后再更新点。

我们保留输入的 v，e，f 称呼。同时将新生成的点叫做 face，edge 等。

在开始时还要对文件进行读入转化操作，可以使用字符串处理的思想处理数据。

本实验的主要思路如下：

1、生成 face 点：

每个 face 点都是 f 中的一个重心点，我们将新生成的 face 点的索引添加到用于生成 face 点的周围 3 或 4 个 v 点中，在我的实现中使用了 set 来存储新的 face 点。

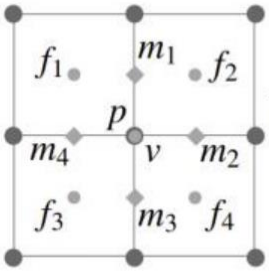
2、生成 edge 点：

每个 edge 点都是由两个 v 点和两个 face 点生成的。因为 f 中保留了完整封闭图形的边点关系，所以再次遍历所有的 f，找到 f 中的每两个相邻点，再找到这两个相邻点中存储 face 点的集合的交集，这个交集中含有的两个点就是我们要生成的 edge 点的最近的两个 face 点，使用生成公式进行生成。

3、更新 v 点

V 点是我们原先输入的顶点，当我们生成完所有的 face 点和 edge 点后就可以更新顶点了。按照这个点是否是一个奇异点更新公式分为了以下的两个：

对于非奇异点：



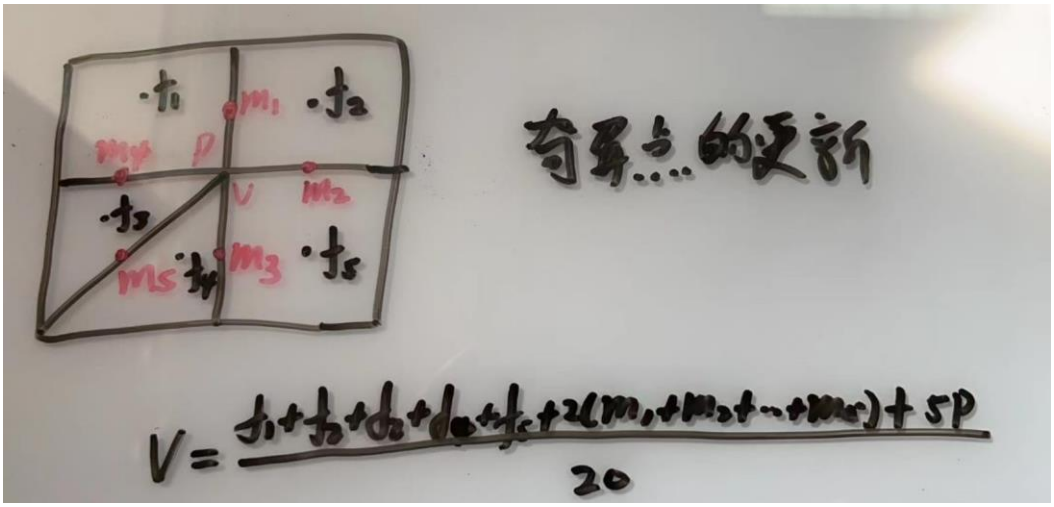
Vertex point

$$v = \frac{f_1 + f_2 + f_3 + f_4 + 2(m_1 + m_2 + m_3 + m_4) + 4p}{16}$$

m midpoint of edge

p old "vertex point"

对于奇异点：



更新后的 v 称作 vnew

4、生成新的 f 与 e

f 是存储一个完整封闭图形的数据结构，因为我们要细分，所以细分后的所有封闭图形也要重新存储。我们遍历所有的原始 f，按照 face，edge1，vnew，edge2 的顺序生成新的 f。存入到 f 数组中。同时，也将所有的新连边生成，存入到 e 数组中。

5、生成所有的点，边结束后，重新绘制点边图。

实验步骤与实验结果：

```
#include <chrono>
#include <iostream>
#include <opencv2/opencv.hpp>
#include <cmath>
#include <vector>
#include <cstring>
#include <fstream>
#include <chrono>
#include <set>
using namespace cv;

struct V{
    int vid;
    int vs2_id;
    double x;
    double y;
    std::set<int> flist;
    std::set<int> elist;
    bool operator<(const V A)const{
        if(this->x == A.x) return this->y < A.y;
        else return this->x < A.x;
    }
};


struct E{
    E(){v1 = 0; v2 = 0;}
    E(int v11, int v21){v1 = v11; v2 = v21;}
    int v1;
    int v2;
};

int find_same_e(V v1, V v2){
    for(auto it1 = v1.elist.begin(); it1 != v1.elist.end(); it1++){
        for(auto it2 = v2.elist.begin(); it2 != v2.elist.end();
it2++){
            if(*it1 == *it2) return *it2;
        }
    }
}

struct F{
```

```
    std::vector<int> vs;
};
//在程序中存储数据:

std::vector<V> vs; // 所有顶点
std::vector<E> es; // 所有边
std::vector<F> fs; // 所有面

std::vector<V> vs2; // 所有顶点 2
// std::vector<E> es2; // 所有边 2
std::vector<F> fs2; // 所有面 2

std::vector<V> vs3; // 所有顶点 3

// int v_id = 0, e_id = 0, f_id = 0;
double s_to_math(std::string &s){// it will change s'
    int del = 0;
    while(del < s.size() && (s[del] == ' ')||(s[del] >= 'a' &&
s[del] <= 'z')||(s[del] == '\n'))del++;
    double ans = 0; double sym = 0;//check point
    int del2 = 0;
    for(int i = del; i < s.size(); i++){
        del2 =  i;
        if(s[i] == ' ' || s[i] == '\n') break;
        if(s[i] == '.'){sym  = 0.1; continue;}
        if(sym == 0){
            ans = ans * 10; ans += s[i] - '0';
        }
        else {
            ans = ans + (s[i] - '0')* sym;
            sym = sym * 0.1;
        }
    }
    std::string temps;
    for(int i = del2 + 1; i < s.size(); i++)temps += s[i];
    s = temps;
    return ans;
}

int main(){
    int changetimes = 2;
    std::ifstream infile;
    infile.open("data.txt", std::ios::in);
    // 检查文件是否成功打开
```

```
    if (!infile.is_open()){
        std::cout << "无法打开文件。" << '\n';
        return 1;
    }
    std::string sline;
    while(getline(infile, sline)){
        if(sline[0] == 'v'){
            V temp;
            temp.x = s_to_math(sline);
            temp.y = s_to_math(sline);
            // temp.v_id = v_id;
            vs.push_back(temp);
        }
        else if(sline[0] == 'e'){
            E temp;
            temp.v1 = (int)s_to_math(sline) - 1;
            temp.v2 = (int)s_to_math(sline) - 1;
            // temp.e_id = e_id;
            es.push_back(temp);
        }
        else if(sline[0] == 'f'){
            F temp;
            int num;
            int countnum = 0;
            for(auto it = sline.begin(); it != sline.end(); it++){
                if(*it == ' ') countnum++;
            }
            for(int j = 1; j <= countnum; j++){
                int num = s_to_math(sline);
                temp.vs.push_back(num - 1);
            }
            fs.push_back(temp);
        }
        else printf("No such data!\n");
    }
    // 关闭文件
    infile.close();

    cv::Mat img = Mat::zeros(Size(800, 800), CV_8UC3);
    img.setTo(255);              // 设置屏幕为白色
    for(auto it = es.begin(); it != es.end(); it++){
        // printf("it->v1\n%d", it->v1);
        cv::Point p1(230*vs[it->v1].x, 230*vs[it->v1].y);
        cv::Point p2(230*vs[it->v2].x, 230*vs[it->v2].y);
```

```
        cv::line(img, p1, p2, cv::Scalar(0, 0, 255), 1);
        cv::circle(img, p1, 3, cv::Scalar(0, 255, 0), -1);
        cv::circle(img, p2, 3, cv::Scalar(0, 255, 0), -1);
    }

    cv::imshow("画板", img);
    printf("in");
    for(int ch = 1; ch <= changetimes; ch++){
        for(auto it = fs.begin(); it != fs.end(); it++){
            int countpoints = it->vs.size();
            V temp; temp.x = 0; temp.y = 0;
            int resentindex = vs2.size();
            for(int j = 0; j < countpoints; j++){
                temp.x += vs[it->vs[j]].x;
                temp.y += vs[it->vs[j]].y;
                vs[it->vs[j]].flist.insert(resentindex);
            }
            temp.x = 1.0 * temp.x / countpoints; temp.y = 1.0 *
temp.y / countpoints;
            vs2.push_back(temp);
        }

        for(auto it = vs2.begin(); it != vs2.end(); it++){
            cv::Point p1(230*it->x, 230*it->y);
            cv::circle(img, p1, 3, cv::Scalar(0, 255, 0), -1);
        }
//        if(ch == 2)return 0;

        int recordsize = vs2.size();
        printf("outfcreate\n");
        for(auto it = es.begin(); it != es.end(); it++){
            int countsame = 0;
            V fnode[3];// drap temp[0]
            for(auto it2 = vs[it->v1].flist.begin(); it2 !=
vs[it->v1].flist.end(); it2++){
                if(vs[it->v2].flist.find(*it2) !=
vs[it->v2].flist.end()){
                    countsame++;
                    fnode[countsame] = vs2[*it2];
                }
            }
            if(countsame < 2){
                // continue;
```

```
                V temp; temp.x = 1.0*(vs[it->v1].x +
vs[it->v2].x)/2;
                temp.y = 1.0*(vs[it->v1].y + vs[it->v2].y)/2;
                int resentindex = vs2.size();
                vs[it->v1].elist.insert(resentindex);
                vs[it->v2].elist.insert(resentindex);
                vs2.push_back(temp);
            }
            else if(countsame == 2){
                V temp; temp.x = 1.0*(vs[it->v1].x + vs[it->v2].x +
fnode[1].x + fnode[2].x)/4;
                temp.y = 1.0*(vs[it->v1].y + vs[it->v2].y +
fnode[1].y + fnode[2].y)/4;
                int resentindex = vs2.size();
                vs[it->v1].elist.insert(resentindex);
                vs[it->v2].elist.insert(resentindex);
                vs2.push_back(temp);
            }
            else printf("Wrong same f point\n");
        }
        // if(ch == 2)return 0;

        for(int i = recordsize; i < vs2.size(); i++){
            cv::Point p1(230*vs2[i].x, 230*vs2[i].y);
            cv::circle(img, p1, 3, cv::Scalar(255, 255, 0), -1);
        }
        //update point
        for(auto it = vs.begin(); it != vs.end(); it++){
            int countf_num = 0;
            int division = 0;
            V tempV = *it;
            double tempx = 0,  tempy = 0;
            for (auto it2 = it->flist.begin(); it2 !=
it->flist.end(); it2++){
                countf_num++; division++;
                tempx += vs2[*it2].x;  tempy += vs2[*it2].y;
            }
            for (auto it2 = it->elist.begin(); it2 !=
it->elist.end(); it2++){
                tempx += 2*vs2[*it2].x;  tempy += 2*vs2[*it2].y;
                division+=2;
            }
            tempV.x = 1.0*(tempx + tempV.x * countf_num)/(countf_num
+ division);
```

```
            tempV.y = 1.0*(tempy + tempV.y * countf_num)/(countf_num
+ division);
            vs2.push_back(tempV);
            it->vs2_id = vs2.size() - 1;
            vs3.push_back(tempV);
        }

        for(auto it = vs3.begin(); it != vs3.end(); it++){
            cv::Point p1(230*it->x, 230*it->y);
            cv::circle(img, p1, 3, cv::Scalar(0, 0, 0), -1);
        }
            cv::imshow("画板", img);
            waitKey(5000);
        // reconnext edge
        vs.clear();
        es.clear();
        // for(auto it = vs2.begin(); it != vs2.end(); it++){
        //     cv::Point p1(230*it->x, 230*it->y);
        //     cv::circle(img, p1, 3, cv::Scalar(0, 0, 0), -1);
        // }
        std::set<V>SET;// check vs have the index or not
        for(auto it = fs.begin(); it != fs.end(); it++){
            int countf_index = it->vs.size();
            V fnode[5];// correspond real node
            int f_countnum = it->vs.size();
            for(int i = 1; i <= f_countnum; i++){
                fnode[i] = vs2[vs[it->vs[i-1]].vs2_id];
                fnode[i].vid = vs[it->vs[i-1]].vs2_id;
            }
            int f_index = 0;
            int e_index[5]; for(int i = 0; i < 5; i++)e_index[i] =
0;
            if(f_countnum == 3){
                for(auto it1 = fnode[1].flist.begin(); it1 !=
fnode[1].flist.end(); it1++){
                    for(auto it2 = fnode[2].flist.begin(); it2 !=
fnode[2].flist.end(); it2++){
                        for(auto it3 = fnode[3].flist.begin(); it3 !=
fnode[3].flist.end(); it3++){
                            if(fnode[2].flist.find(*it3) !=
fnode[2].flist.end() && fnode[1].flist.find(*it3) !=
fnode[1].flist.end()){
                                f_index = *it3;// f_index
                            }
```

```
                        }
                    }
                }
                e_index[1] = find_same_e(fnode[1], fnode[2]);
                e_index[2] = find_same_e(fnode[2], fnode[3]);
                e_index[3] = find_same_e(fnode[3], fnode[1]);
                F f1, f2, f3;
                f1.vs.push_back(f_index);f1.vs.push_back(e_index[3])
;f1.vs.push_back(fnode[1].vid);f1.vs.push_back(e_index[1]);
                f2.vs.push_back(f_index);f2.vs.push_back(e_index[1])
;f2.vs.push_back(fnode[2].vid);f2.vs.push_back(e_index[2]);
                f3.vs.push_back(f_index);f3.vs.push_back(e_index[2])
;f3.vs.push_back(fnode[3].vid);f3.vs.push_back(e_index[3]);
                fs2.push_back(f1);fs2.push_back(f2);fs2.push_back(f3
);
                es.push_back(E(fnode[1].vid, e_index[1]));
es.push_back(E(e_index[1], fnode[2].vid));
es.push_back(E(fnode[2].vid, e_index[2]));
es.push_back(E(e_index[2], fnode[3].vid));
es.push_back(E(fnode[3].vid, e_index[3]));
es.push_back(E(fnode[1].vid, e_index[3]));
                es.push_back(E(f_index, e_index[1]));
es.push_back(E(f_index, e_index[2])); es.push_back(E(f_index,
e_index[3])); es.push_back(E(f_index, e_index[3]));
            }
            else if(f_countnum == 4){
                for(auto it1 = fnode[1].flist.begin(); it1 !=
fnode[1].flist.end(); it1++){
                    for(auto it2 = fnode[2].flist.begin(); it2 !=
fnode[2].flist.end(); it2++){
                        for(auto it3 = fnode[3].flist.begin(); it3 !=
fnode[3].flist.end(); it3++){
                            for(auto it4 = fnode[4].flist.begin();
it4 != fnode[4].flist.end(); it4++)
                            if(fnode[3].flist.find(*it4) !=
fnode[3].flist.end() && fnode[2].flist.find(*it4) !=
fnode[2].flist.end() && fnode[1].flist.find(*it4) !=
fnode[1].flist.end()){
                                f_index = *it4;// f_index
                            }
                        }
                    }
                }
                e_index[1] = find_same_e(fnode[1], fnode[2]);
```

```
                e_index[2] = find_same_e(fnode[2], fnode[3]);
                e_index[3] = find_same_e(fnode[3], fnode[4]);
                e_index[4] = find_same_e(fnode[4], fnode[1]);
                F f1, f2, f3, f4;
                f1.vs.push_back(fnode[1].vid);f1.vs.push_back(e_inde
x[1]);f1.vs.push_back(f_index);f1.vs.push_back(e_index[4]);
                f2.vs.push_back(fnode[2].vid);f2.vs.push_back(e_inde
x[2]);f2.vs.push_back(f_index);f2.vs.push_back(e_index[1]);
                f3.vs.push_back(fnode[3].vid);f3.vs.push_back(e_inde
x[3]);f3.vs.push_back(f_index);f3.vs.push_back(e_index[2]);
                f4.vs.push_back(fnode[4].vid);f4.vs.push_back(e_inde
x[4]);f4.vs.push_back(f_index);f4.vs.push_back(e_index[3]);
                fs2.push_back(f1);fs2.push_back(f2);fs2.push_back(f3
);fs2.push_back(f4);
                es.push_back(E(fnode[1].vid,
e_index[1]));es.push_back(E(e_index[1],
f_index));es.push_back(E(f_index,
e_index[4]));es.push_back(E(fnode[1].vid, e_index[4]));
                es.push_back(E(fnode[2].vid,
e_index[2]));es.push_back(E(e_index[2],
f_index));es.push_back(E(fnode[2].vid, e_index[1]));
                es.push_back(E(fnode[3].vid,
e_index[3]));es.push_back(E(e_index[3],
f_index));es.push_back(E(fnode[3].vid, e_index[2]));
                es.push_back(E(fnode[4].vid,
e_index[4]));es.push_back(E(fnode[4].vid, e_index[3]));
            }
            else printf("Wrong coset_f_num not 3 and not 4\n");
        }
    img.setTo(cv::Scalar(255, 255, 255));
    for(auto it = es.begin(); it != es.end(); it++){
        // printf("it->v1\n%d", it->v1);
        cv::Point p1(230*vs2[it->v1].x, 230*vs2[it->v1].y);
        cv::Point p2(230*vs2[it->v2].x, 230*vs2[it->v2].y);
        cv::line(img, p1, p2, cv::Scalar(0, 0, 255), 1);
        cv::circle(img, p1, 3, cv::Scalar(0, 255, 0), -1);
        cv::circle(img, p2, 3, cv::Scalar(0, 255, 0), -1);
    }
        fs.clear(); vs.clear();
        // for(auto it = )
        fs = fs2;
        vs = vs2;

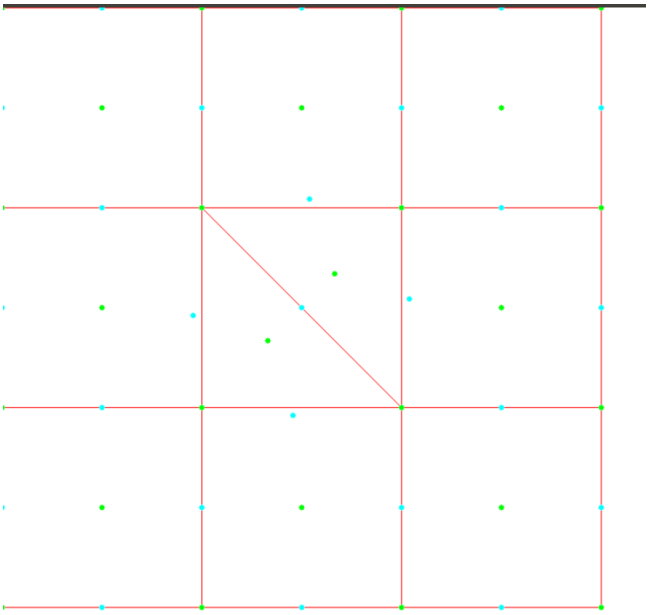
        // fs2.clear(); vs2.clear();
```

```
        // std::swap(fs, fs2); fs2.clear();
        // std::swap(vs, vs2); vs2.clear();
        cv::imshow("画板", img);
        waitKey(5000);
    }
    // cv::imshow("画板", img);
    // waitKey(0);
    return 0;
}
```

实验效果：

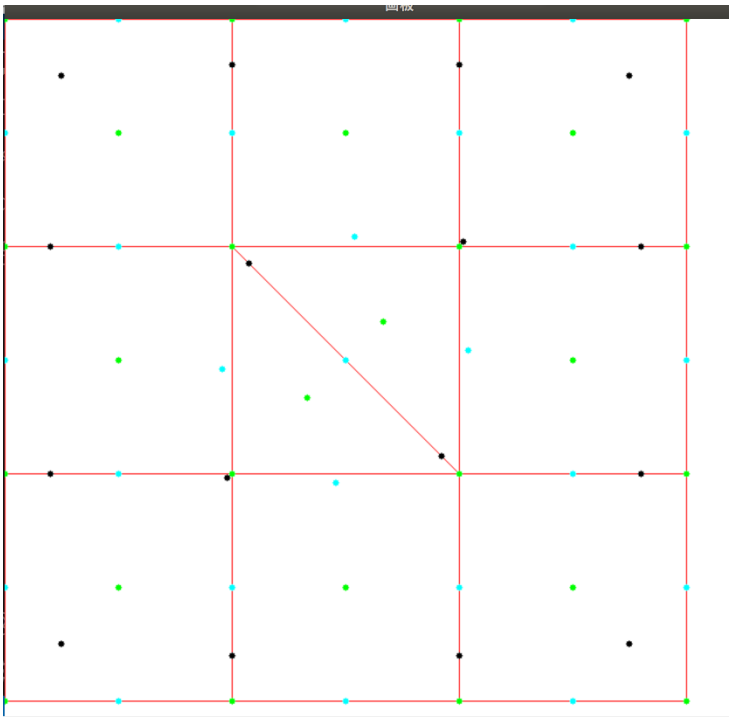
第一次迭代生成 face 点和 edge 点：

绿色的是 face 点，蓝色的是 edge 点

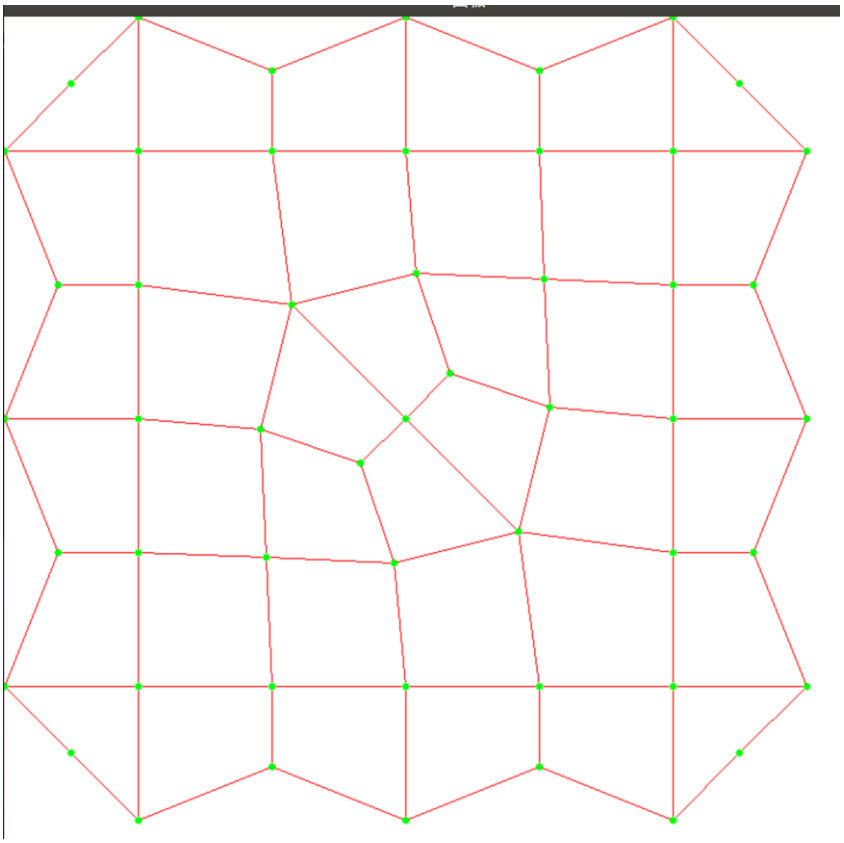


第一次迭代更新 v 点

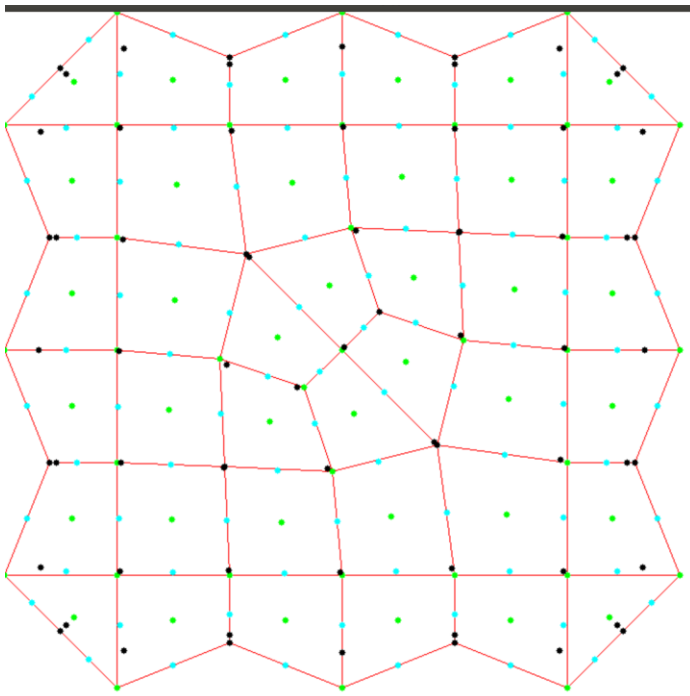
黑色的点为 vnew 点



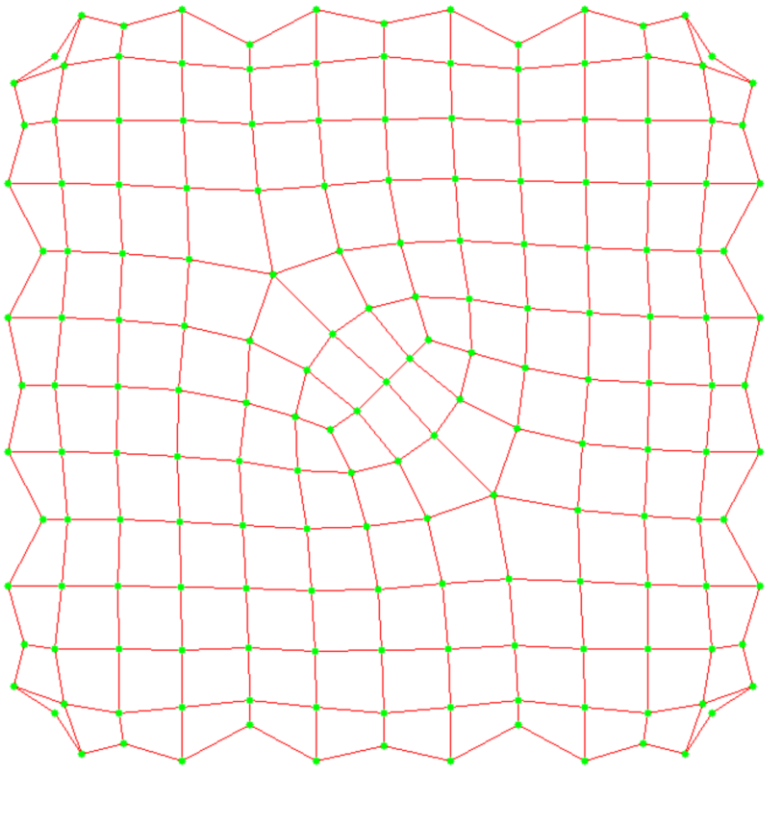
第一次迭代效果图：



第二次迭代生成点：



第二次迭代效果图：



实验中存在的问题及解决：

1、要生成 edge 点需要借助之前生成的 face 点，怎样找到对应的两个 face 点呢？

答：使用 set 将新生成的 face 点的索引存入周围所有顶点中，这样找到同一条边的两个顶点时，两个含有 face 点的 set 做一个交集就可以找到对应的 face 点了。

2、怎么样在循环中持续迭代？

答：每次迭代结束后更新 f，即将细分的新的封闭图形存储下来，这样点边的关系也被存储了下来。

3、我的代码理论上可以无限细分，但是实际上在第三次细分的时候会出现 segment fault。我推断是由于内存管理出现问题。